

CH 3.2: TRANSACTIONS

Contents

Types of Transactions	1
A. Based on Transaction Modeling Approach.....	1
B. Based on Execution Scope and Structure	2

Types of Transactions

A. Based on Transaction Modeling Approach

1. General Model

The most flexible transaction model.

- A transaction can consist of **multiple sub-transactions**.
- Each sub-transaction can run at **different sites**.
- Sub-transactions can **spawn further sub-transactions**.
- **Use Case:** Highly complex distributed applications with dynamic behavior.

2. Two-Step Model

- A restricted version of the general model with **two levels**.
- One **root transaction** that initiates multiple **independent sub-transactions**.
- Sub-transactions **do not create further transactions**.
- **Use Case:** Easier to manage, suitable for moderately complex systems.

3. Restricted Model

- A simpler model where all operations are at **one level**.
- The transaction is split into sub-transactions **without nesting**.
- Sub-transactions must **run sequentially** or follow a strict order.
- **Use Case:** Simple workflow systems with clear task ordering.

4. Restricted Two-Step Model

- Combines limitations of the **Restricted** and **Two-Step** models.
- Only **two levels** allowed (main + sub-transactions).
- Sub-transactions are **restricted**: no further sub-task creation, and limited communication.
- **Use Case:** Environments needing **predictability and control**, like workflows or sequential processing.

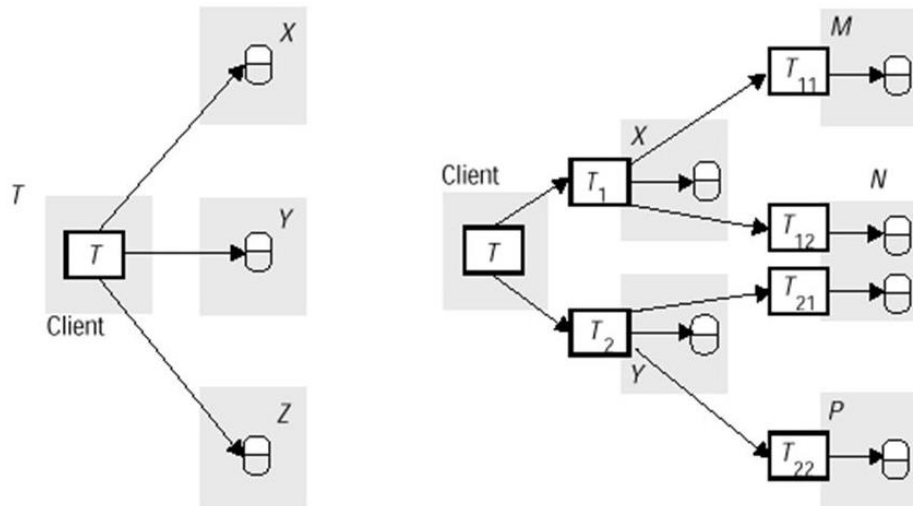
5. Action Model

- A specialized model for **long-lived or cooperative transactions**.
- Transactions are broken into **actions** (small, recoverable units).
- Supports **undo/redo** via compensating actions.
- **Flexible execution and recovery**.
- **Use Case:** Business processes, collaborative work, or distributed workflows (e.g., e-commerce, travel booking).

B. Based on Execution Scope and Structure

1. Flat Transaction

- A basic, single-level transaction that runs from start to finish without internal sub-transactions.
- All operations are **executed sequentially**.
- It either **commits fully** or **rolls back** completely (atomic).
- **Simplest** type of transaction.
- **Example:** A banking system where money is transferred from Account A to Account B — debit and credit happen in the same flat transaction.



Flat and nested transaction

2. Nested Transaction

- A transaction that contains **sub-transactions**, forming a hierarchy.
- Parent transaction spawns **child transactions**.
- Sub-transactions can **commit independently**, but final commit of the parent determines overall success.
- Supports **parallelism and modularity**.
- **Example:** An e-commerce order process may have sub-transactions for:
 - Checking inventory
 - Processing payment
 - Arranging shipping
 - Each of these can be treated as sub-transactions of the main order transaction.

3. Distributed Transaction

- **Definition:** A transaction that accesses **multiple databases or sites** in a distributed system.
- Operates over a **network of databases**.
- Requires coordination (e.g., **2-phase commit protocol**) to ensure atomicity across sites.
- Can be **flat** or **nested**.
- **Example:** A mobile wallet transaction updates balances on servers located in two different cities/databases.

4. Global Transaction

- A transaction **initiated by a global application** and may include multiple **local transactions** on different sites.
- Managed by a **global transaction manager**.
- Ensures **global consistency** across participating systems.
- Typically composed of multiple distributed transactions.

- **Example:** A cloud-based ERP system executing a company-wide monthly financial closure across all departments.

5. Local Transaction

- A transaction that executes **only on one site/database**.
- No interaction with other sites.
- Simpler to execute and manage.
- Does not require distributed coordination protocols.
- **Example:** A university student updating their course registration in a single university database.

Transaction Type	Scope	Hierarchy	Complexity	Use Case
Flat	Single site or multiple	No sub-tasks	Low	Simple tasks like bank transfer
Nested	Single or multiple sites	Has sub-tasks	Medium	Modular tasks like order processing
Distributed	Multiple sites/databases	May be flat/nested	High	Multi-location ATM transaction
Global	Multiple sites/databases	Oversees locals	High	Enterprise-level apps like SAP
Local	One database/site only	Flat	Low	Simple local DB operations